

Parte 1 - Teoria

1. O que é a GLSL? Quais os dois tipos de shaders obrigatórios no pipeline programável e o que eles processam?

GLSL (OpenGL Shading Language) é a linguagem de programação utilizada para escrever shaders em OpenGL.

Os dois shaders obrigatórios são:

Vertex Shader: processa cada vértice individualmente, transformando suas posições (por exemplo, do espaço do modelo para o espaço de projeção).

Fragment Shader: calcula a cor de cada fragmento (pixel candidato à exibição na tela), aplicando texturas, iluminação e efeitos visuais.

2. O que são primitivas gráficas? Como fazemos o armazenamento dos vértices na OpenGL?

Primitivas gráficas são formas básicas como pontos, linhas e triângulos, usadas para compor imagens 2D/3D.

Na OpenGL, os vértices são armazenados em Vertex Buffer Objects (VBOs) e organizados por Vertex Array Objects (VAOs).

3. Explique o que é VBO e VAO, e como se relacionam.

VBO (Vertex Buffer Object): armazena dados dos vértices (posições, cores, normais etc.).

VAO (Vertex Array Object): guarda as configurações sobre como os dados nos VBOs são interpretados (por exemplo, qual parte do buffer representa posições ou cores).

A relação entre eles é que o VAO aponta para um ou mais VBOs e especifica como usar seus dados.

4. Como são feitas as chamadas de desenho na OpenGL? Para que servem as primitivas de desenho?

As chamadas são feitas com funções como `glDrawArrays()` ou `glDrawElements()`.

As primitivas de desenho indicam como os vértices devem ser conectados — por exemplo, `GL_TRIANGLES`, `GL_LINES`, `GL_POINTS`.

Elas controlam como a GPU forma as figuras a partir dos dados dos vértices.